
CryptoCore-16

16-bit Cryptographic Coprocessor in VHDL

Meet Our Team

Hatem Ayman

Project coordination,
Combinational Logic Block

Eyad Islam

Arithmetic Logic Unit,
Project report

Elsayed Abdul Raouf

Non-Linear Lookup Table,
Top-Level coprocessor

Alaa Muhammad

Shifter Unit,
Top-Level schematic

Amal Ragab

Register File,
Top-Level simulation

Why a cryptographic coprocessor?

Software Bottleneck

AES/DES/ECC in software are severe CPU bottlenecks — thousands of encrypted connections require sequential SubBytes, ShiftRows, and MixColumns per round.

Timing Side-Channel Attacks

Cache and branch predictor variability in software S-Box lookups expose measurable timing differences — exploited in known attacks.

Hardware Advantages

Constant-time execution, parallel datapaths, and a dedicated instruction set eliminate both performance and security bottlenecks simultaneously.

Key Goal

1 instruction per clock cycle throughput with **zero timing variation** — regardless of operand values or operation type.

Instruction set Architecture- ISA

Instruction Word Format: [CTRL[3:0] | Ra[3:0] | Rb[3:0] | Rd[3:0]] — 16-bit fixed-width

CTRL	Mnemonic	Unit	Operation Description	Write-Back
0000	ADD	ALU	ABUS + BBUS	✓
0001	SUB	ALU	ABUS − BBUS	✓
0010	AND	ALU	ABUS & BBUS	✓
0011	OR	ALU	ABUS BBUS	✓
0100	XOR	ALU	ABUS ⊕ BBUS	✓
0101	NOT	ALU	~ABUS	✓
0110	MOV	ALU	ABUS	✓
0111	NOP	ALU	No operation — write DISABLED	✗
1000	ROR8	Shifter	Rotate BBUS right 8 bits (byte swap)	✓
1001	ROR4	Shifter	Rotate BBUS right 4 bits (nibble rotation)	✓
1010	SLL8	Shifter	Shift BBUS left 8 bits (zero fill)	✓
1011	LUT	S-Box	S-Box substitution on ABUS[7:0]	✓

Modularity

Layer 1 — Compute Units

Purely Combinational — No Clock

- `ALU.vhd` — 8 operations
- `shifter.vhd` — 3 shift/rotate ops
- `LUT.vhd` — dual 4-bit S-Box substitution

Layer 2 — Control

Structural Integration

- `combinational_logic_block.vhd` — instantiates ALU, Shifter, LUT
- All THREE units compute in parallel every cycle
- 2-level MUX routes result to `RES[15:0]`

Layer 3 — Datapath & Top-Level

Sequential Logic

- `register.vhd` — 16×16-bit register file, async read, sync write
- `crypto_core16.vhd` — top-level entity with 1-stage input pipeline



Unit Testbenches (one per entity): `ALU_tb` | `shifter_tb` | `LUT_tb` | `register_tb` | `combinational_logic_block_tb` | `crypto_core16_tb`

Arithmetic Logic Unit



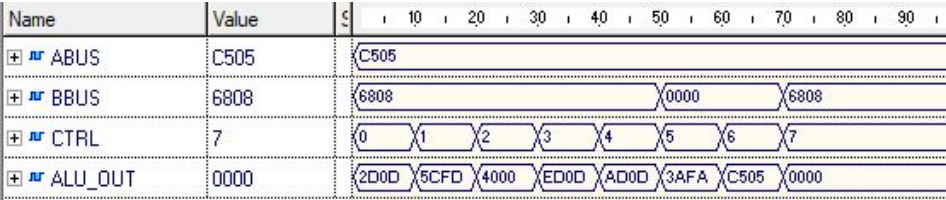
Interface & Implementation

Inputs: ABUS[15:0], BBUS[15:0], CTRL[3:0]

Output: ALU_OUT[15:0]

- All 8 operations decoded with a single case statement — no priority encoding needed, each opcode maps to one result.
- Arithmetic uses ieee.numeric_std unsigned type — synthesis tool selects optimal carry-chain
- CTRL(3)='0' dedicates the entire lower opcode half (0000–0111) to ALU — a single-bit MUX decision at the top level.

Simulation Waveform Reference



Testbench Results

ABUS=0xC505, BBUS=0x6808

Op	Result	Status
ADD	0x2D0D	✓ Verified
SUB	0x5CFD	✓ Verified
AND	0x4000	✓ Verified
OR	0xED0D	✓ Verified
XOR	0xAD0D	✓ Verified
NOT	0x3AFA	✓ Verified
MOV	0xC505	✓ Verified
NOP	0x0000	✓ No write

Shifter Unit

Interface & Implementation

Input: BBUS[15:0] (ABUS ignored), CTRL[3:0]
Output: SHIFT_OUT

Activation:

Triggered when CTRL(3)='1' (Opcodes: 1000–1010)

- **ROR8**: Rotate right by 8 bits
- **ROR4**: Rotate right by 4 bits
- **SLL8**: Logical shift left by 8 bits

Testbench Results BBUS=0x1234

Operation	Result	Behavior
ROR8	0x3412	Bytes swap; no loss
ROR4	0x4123	Nibble rotation; no loss
SLL8	0x3400	Zero-filled; info lost

Simulation Waveform Reference

Name	Value	Time (ns)
B_Bus	1234	0 10 20 30 40 50 60 70 80 90
CTRL_shifter	0	0 9 A 0
shifter_result	0000	3412 4123 3400 0000

Non-Linear Lookup Table (S-Box)

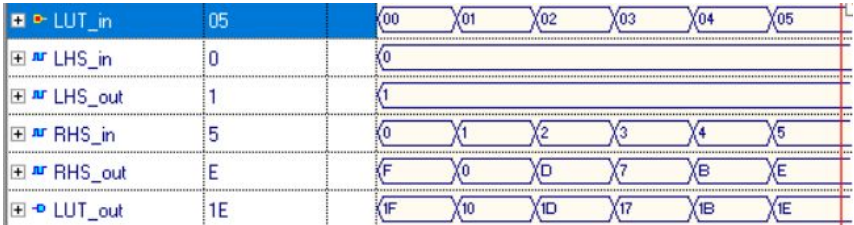
Interface and implementation

- Input: `ABUS[7:0]` only; the upper byte `ABUS[15:8]` passes through unchanged
- Activated only by `CTRL="1011"` — the dedicated LUT opcode
- Architecture uses dual 4-bit S-Boxes: the lower byte is split into two nibbles
 - `LHS_in = ABUS[7:4] → SBOX_1 → LHS_out`
 - `RHS_in = ABUS[3:0] → SBOX_2 → RHS_out`
 - Final output: `ABUS[15:8]` concatenated with (`LHS_out` & `RHS_out`)

Testbench Results `ABUS=0x__05`

Upper byte	<code>0x__</code>	Passes through unchanged
Lower byte	<code>0x05</code>	Splits for S-Box
LHS_in	<code>0x0</code>	Input to SBOX_1
RHS_in	<code>0x5</code>	Input to SBOX_2
Result	<code>0x1E[combined _output]</code>	Final output format

Simulation Waveform Reference



Combinational Logic Block - Control Logic

Structural Binding

Single entity instantiating `ALU_UNIT`, `SHIFTER_UNIT`, and `LUT_UNIT`. All driven concurrently from `ABUS`, `BBUS`, and `CTRL`.



Key Insight

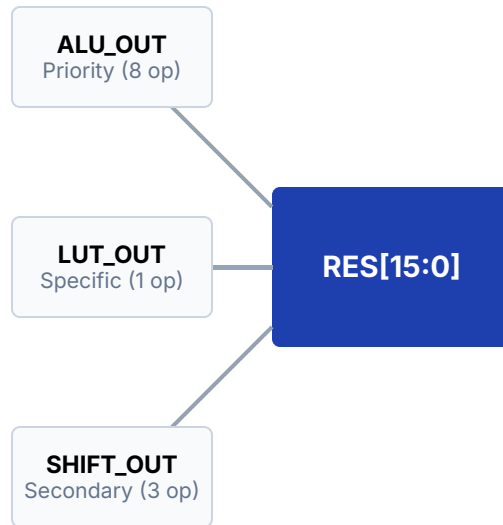
Units compute for every change of `ABUS`, `BBUS`, `CTRL`. The MUX selects the result to route—no enable signals or gating used.



Synthesis Note

Implemented as 16 parallel 2-input MUX chains (one per output bit), visible as `Mux0–Mux15` in RTL.

Priority MUX Logic (2-Level)



Level 1: `CTRL(3)='0'` selects `ALU_OUT`

Level 2: `CTRL="1011"` selects `LUT_OUT`

Default: All others select `SHIFT_OUT`

Register File

Architecture: 256-bit Total Storage | 16 registers × 16-bit wide

Dual Asynchronous Read Ports

$SRCa \leftarrow REG_FILE[Ra] \rightarrow ABUS$

$SRCb \leftarrow REG_FILE[Rb] \rightarrow BBUS$

Combinational concurrent signal assignment — reads never wait for clock.

Single Synchronous Write Port

Triggered on `rising_edge(clock)`

- Reset clears all 16 registers to `0x0000`
- `RdWEn = '1'` writes $REG_FILE[Rd] \leftarrow RES$

Design Choice: Async reads + sync writes = zero read latency for CPU pipeline. No read-after-write hazard within the same cycle.

Simulation Verification

Test Description

Pre-init values read immediately (no clock)
Sync reset at 20 ns drops both outputs to `0x0000`
Write `0xAAAA` to R3, confirmed via async read
Write `0x1234` to R15, confirmed via async read

Result

✓ Pass
✓ Pass
✓ Pass
✓ Pass

16

Registers

2

Read Ports

1

Write Port

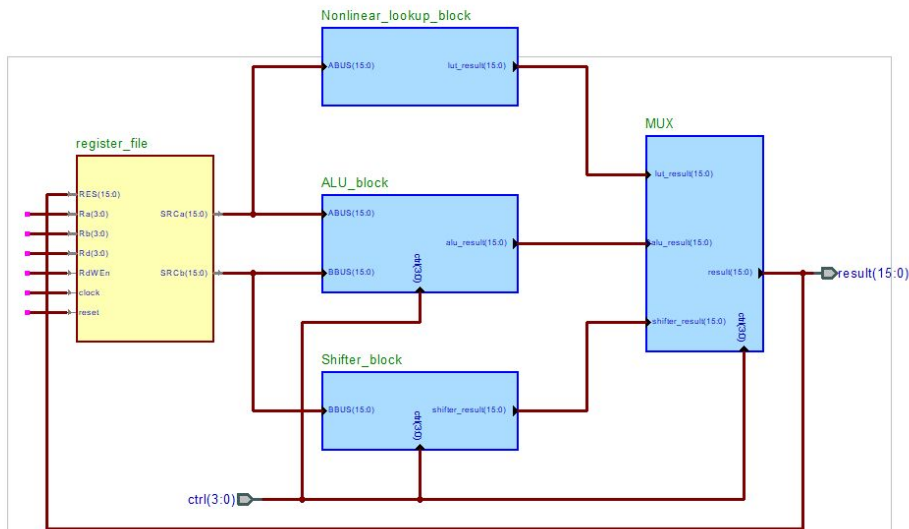
Top-Level schematic

Structural Binding

register_file ↔
structural_combination
al_logic_block

Connected via:

- ABUS, BBUS
(register reads)
- RESULT
(write-back loop)



(C)ALDEC, Inc
2260 Corporate Circle
Henderson, NV 89074



Created: 5/3/2026

Title: Design_Schematic

Top-Level coprocessor

Pipeline Register

- Captures CTRL on every rising edge into `ctrl_tmp` → isolates combinational block from input glitches
- `CTRL="0111"` (NOP) → `RdWEn='0'` (suppress write); all others → `RdWEn='1'`
- On reset: `ctrl_tmp ← "0111"` (NOP), `RdWEn ← '0'` — safe quiescent power-up state

Execution Trace — XOR R1, R8 →

R7

Cycle N	CTRL="0100", Ra=R1, Rb=R8, Rd=R7 captured in ctrl_tmp
ABUS	0xC505 (from R1, async)
BBUS	0x6808 (from R8, async)
ALU	0xC505 XOR 0x6808 = 0xAD0D
MUX	CTRL(3)=0 → selects ALU_OUT
Cycle N+1	RdWEn='1', REG[7] ← 0xAD0D ✓

Top-Level simulation Analysis

12

Instructions

All verified ✓

100

MHz Clock

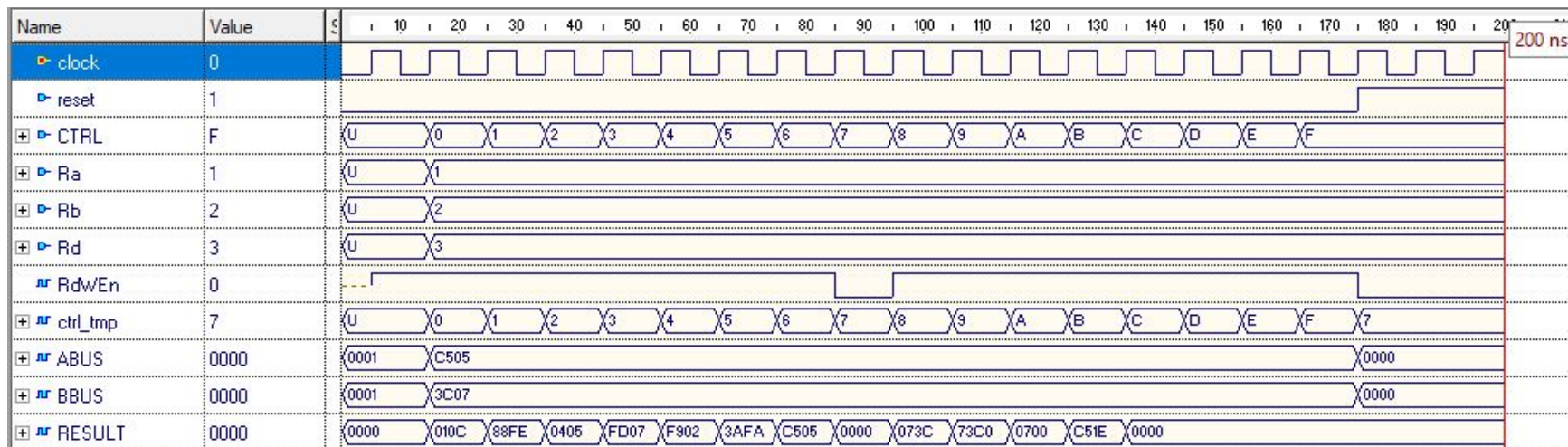
10 ns period

256

LUT Inputs

All simulated ✓

Pipeline Observation: Write-back appears exactly ONE clock cycle after CTRL is applied — confirming correct 1-stage pipeline behavior across all 12 instructions.



Any Questions?





Thank You for Listening

CryptoCore-16
